Guide 5 · Code Reading Drills

A method for understanding any codebase fast, practised on AgriAgent itself

AgriAgent Final Year Project · 20 October 2026

> **Why this guide exists.** Guides 1–4 explain AgriAgent. This one trains a skill: opening code you did not write and understanding it quickly enough to explain, change and defend it, in a viva or an interview. An examiner or interviewer does not mind that AI helped write the code. They check whether you can say what each line does and what breaks if it changes. Every drill below is done **with the terminal open**. Reading these pages without running anything trains nothing.

1 The method

Use these nine habits on any codebase, in roughly this order.

Habit	What you do
1. Start at the entry point	Find where execution begins (`main`, the CLI script, the app) and follow **one real input** to the output. Never read files top to bottom.
2. Follow the data	Find the object passed between the steps and what each step adds to it. In AgriAgent it is the `state` dictionary: know its keys and you know the system.
3. Outline before reading	`rg -n "^def |^class |    def " file.py` shows a file's shape in seconds. Then read only the function you need.
4. Predict, then check	Write down what you expect **before** running anything. A wrong guess shows you exactly what you did not understand.
5. Stop inside a running program	Put `breakpoint()` in a function, run it, and look at real values. Watching values beats reading code about ten times over.
6. Read the tests as the specification	A test states what the code must do, with a concrete input and output.
7. Ask git why	`git log -S "name"` finds the commit that added a line; its message usually says why.
8. Break it on purpose	Change one value and run the tests. Predict which test fails. If **none** fails, you have found a gap in the tests.
9. Explain it in two sentences	Out loud, without looking. If you can't, you don't know it yet.

2 Tool sheet

Command	Use
`rg -n "word" src`	Every line containing a word, with file and line number.
`rg -n "function_name\("`	Every place a function is **called**: who uses this?
`rg -n "^def |^class |    def " f.py`	Outline of one file.

`breakpoint()`	Stops the program at that line and opens the debugger.
`p expr` · `pp expr`	Debugger: print a value (`pp` prints it neatly).
`n` · `s` · `c` · `w` · `l` · `q`	Debugger: next line, step into a call, continue, where am I (call stack), list code around here, quit.
`git log --oneline -S "text"`	Commits that added or removed `text`.
`git show <commit>`	What one commit changed, with its message.
`git blame -L 60,70 f.py`	Who last changed lines 60–70, and in which commit.
`PYTHONPATH=. python -m unittest discover -s tests -k name`	Run only the tests whose names contain `name`.

Before every session, from the repository root in fish: `source ~/.venvs/agriagent/bin/activate.fish`. Then `python` below is the project's Python; always run from the repository root with `PYTHONPATH=.`. Mock mode needs no GPU and runs in seconds.

3 Rules for the study sessions

- **You type every command.** Claude does not run them for you.
- **You explain first;** Claude corrects. No lecture before you have tried.
- **Every session ends with a small change you make yourself,** and the test suite must stay green.
- Undo every deliberate breakage afterwards with `git checkout <file>`, and check `git status` shows nothing changed before you stop.

4 Drill 1 · Map the pipeline (30 min) · `src/workflow.py`

Step	Do
1	`rg -n "^def |^class |    def " src/workflow.py`. From the names alone, **write down the order you think the nodes run in.**
2	Find `def run(` and read only that function. Was your order right? Where is the branch that decides whether System 2 runs? Why can the refine step run more than once?
3	Add `breakpoint()` as the first line of `spray_map_node`, then run `PYTHONPATH=. python scripts/run_live_pipeline.py --mock`. At the `(Pdb)` prompt type `p list(state.keys())`. **For each key, which earlier node produced it?**
4	Type `w`: which functions are on the call stack, from the script down to here? Then `c`, and remove the `breakpoint()`.

Explain in two sentences: what `run()` does, and what `merge_node_output` does to `state`.

5 Drill 2 · The safety gate (45 min) · `spray_map_node`, `label_policy`

Step	Do

1	Predict what `label_policy` returns for the prompt `weed . sugar beet plant .` and the labels `weed`, `sugar beet plant`, `weed sugar beet plant`, `plant`. Then check: `PYTHONPATH=. python -c "from src.workflow import label_policy as l; print(l('weed', 'weed . sugar beet plant .'))"` (and the other three).
2	Read `spray_map_node` from the top to the per-mask loop. List, in order, every reason a mask can end up **withheld** instead of sprayed. Check your list against the `reason` strings: `rg -n '"reason"' src/workflow.py`.
3	`rg -n "class DeterministicRouter" src` and read its `route` method. Which two numbers does it compare, against which limits?
4	**Break it:** set `MAX_PLANT_FRACTION = 0.95` in `src/workflow.py`. Predict which test fails, then run the suite. Read that test: what situation does it protect against, and why would a whole-photo mask be dangerous? Undo the change.
5	**Find the gap:** in `__init__`, change `max(0.70, float(min_confidence))` to `float(min_confidence)`. Run the suite. Did anything fail? What could a user now do with the app's slider that they could not before? Undo the change.
6	**Your change:** write a test in `tests/test_spray_export_safety.py` that fails with the step 5 breakage and passes on the real code. (Hint: build a runner with `mock_mode=True, min_confidence=0.3` and look at one attribute.)

Explain in two sentences: why the gate is plain rules rather than a model, and what "fail closed" means here.

6 Drill 3 · The plant rules (45 min) · `src/vision/plant_candidates.py`

Step	Do
1	Outline the file. Read the constants at the top and write one sentence for each: what would a larger value do?
2	Read `classify_plants` only. Draw its steps as boxes (vegetation → gaps closed → patches → crop / weed / unsure). Which step uses the crop masks from SAM 2?
3	Read `tests/test_plant_candidates.py`. For each test, write the rule it checks in plain words.
4	**Find the gap:** set `SMALL_VS_CROP = 0.9`. Predict which test fails, then run the suite. What did happen? What would this change do to real photos (more weeds found, more crop sprayed)? Undo the change.
5	**Your change:** write a test that pins the size rule: a patch at 0.2 × the median crop plant is a weed, one at 0.5 × is not called a weed by size alone.
6	`rg -n "plant_rules" src/workflow.py`: where does the workflow call the classifier, and what label do its zones get on the spray map?

Explain in two sentences: the two facts about young sugar beet the rules rely on, and why they fail on the cotton close-ups.

7 Drill 4 · The metrics (30 min) · `src/evaluation/crop_protection.py`

Step	Do
1	Read the docstring of `spray_metrics`. Without code, compute the three percentages for grid A of task SA-2 in the Team Tasks PDF.

2	Find the test that uses that grid (`rg -n "33.3" tests`). Do your numbers match?
3	Why does a crop plant count as damaged when **any** pixel is sprayed, but a weed counts as hit only when **half** its pixels are? Argue both sides.
4	`git log --oneline -S "spray_footprint"`, then `git show` the commit that fixed known issue K10. What was wrong before, and on which machines?

Explain in two sentences: why "crop pixels sprayed" is the main number, and why the held-out set was used only once.

8 Drill 5 · The escalation agent (30 min) · `src/agents/escalation_agent.py`

Step	Do
1	Read `p_weed`. By hand: Clef Flash (accuracy 0.70) says weed with confidence 0.9, then Groq (accuracy 0.80) says weed with confidence 0.9. Compute P(weed). Is it at least 0.85, so the agent decides "weed"?
2	Read `review_plant`. When does it stop asking sources? What happens when the budget is too small for Groq?
3	Find the test that checks your step 1 case (`rg -n "0.847|0.85" tests/test_escalation_agent.py`).
4	Why cap each answer by the source's **measured** accuracy instead of trusting its stated confidence? Connect this to the result: 69 reviews, 0 weed verdicts.

Explain in two sentences: what the agent may and may not do, and why its result is "safe, not effective".

9 Drill 6 · System 2 and the ICAR table (30 min) · `src/agronomy/`

Step	Do
1	`rg -n "^def " src/agronomy/prescription.py`. Which function builds a prescription, and which one filters by crop (`rg -n "_crop_key" src`)?
2	`git log --oneline -S "Pyrazosulfuron"`, then `git show` the ICAR-check commit. What was dangerous about the old rice row? Read `docs/results/icar_table_check_20261020.md` to confirm.
3	Read `TestIcarTableMatchesBulletin`. Would these tests have caught the old rice row?
4	Read `src/agronomy/farmer_summary.py`: what stops the language model from inventing a dose in the farmer summary?

Explain in two sentences: where herbicide advice comes from, and why never from the model.

10 Drill 7 · Read library code (30 min) · the TC-15 crash

Interviewers like this one: understanding code **you did not write and cannot change**.

1. `git show d66433b`: read the commit message and the diff. What error did real mode hit?
2. Open the library: `rg -n "def post_process_masks" ~/.venvs/agriagent/lib/python3.11/site-packages/transformers/models/sam2/`.
   Read the function signature. Which inputs does it need, and where does our `_hf_segment` get them?
3. Explain the bug in two sentences: what changed in transformers 5.x, and why our old code broke.

11 Drill 8 · A codebase you have never seen (1 h)

The real test of the skill. Pick a small open-source Python project (the `requests` library is a good size), clone it into a folder outside this repository, and in one hour, using only the method above:

1. find the entry point of `requests.get` and follow it to the line that opens the network connection;
2. name the object passed between the layers;
3. find one test that checks a redirect, and explain it.

Time yourself. Repeat with a different project a week later; it should take noticeably less time.

12 How to talk about code in an interview

1. **Start with the purpose,** one sentence: "This function decides which masks may be sprayed."
2. **Then the data:** what comes in, what goes out.
3. **Then the one decision that matters:** "A mask is withheld if its label names the crop or it covers more than half the photo."
4. **Then how you know it works:** "This test checks exactly that case."
5. **If you don't know:** "I'd check by putting a breakpoint here and looking at the value." That is a strong answer, not a weak one.

13 Progress log

Drill	Date	What I got wrong first	Change made
1			
2			
3			
4			
5			
6			
7			
8			